

✓ Hands-On Activity 5 | Chart Type

Name : Mendoza, Roy Gabriel I.

Course Code and Title : CPE 031-CPE21S1 - Visualizations and Data Analysis

Date Submitted : Aug 22 2026

Instructor : Engr. Jimlord Quejado

1. Objectives:

This activity aims to develop the students' ability to visualize and create data graphs.

2. Intended Learning Outcomes (ILOs): By the end of this module the students should be able to demonstrate:

1. Analyze data charts in various use cases.
2. Create effective data visualizations using charts.

3. Discussions:

Selecting the right chart type is important because it determines how clearly the data can be understood. Different charts serve different purposes—bar and column charts are best for comparing categories, line charts highlight trends over time, pie charts show proportions, and scatter or bubble charts reveal relationships between variables. Choosing the correct chart depends on the nature of the data and the problem you want to analyze. A well-selected chart makes patterns more visible, reduces confusion, and helps in drawing accurate insights.

4. Procedures:

Step 1 : Defining the problem using dataset

The data on road accident causes highlights a significant problem in road safety. By looking at the distribution, we can see that some causes occur far more often than others, showing clear patterns in why accidents happen. For example, if one category such as overspeeding or human error makes up the largest portion, it means that many accidents could be prevented by addressing that specific issue. This suggests that the problem is not just random but is linked to certain behaviors or conditions, and solving it will require targeted actions like stricter enforcement of traffic rules, better road designs, or awareness campaigns for drivers.

```
1 #Install pandas
2 !pip install pandas
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.13/dist-packages (from pandas) (2.1.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pandas) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas) (2026.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.2) (1.17.0)
```

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import numpy as np
```

1. Create a local folder titled "CPE 031_SURNAME" that easily to locate.
2. Download the RoadAccidents.csv
3. Save it to the folder created.

```
1 df = pd.read_csv("/content/RoadAccidents.csv")
```

```
1 df
```

Million Plus Cities	Cause category	Cause Subcategory	Outcome of Incident	Count
---------------------	----------------	-------------------	---------------------	-------

	million plus cities	cause category	cause subcategory	outcome of incident	count
0	Agra	Traffic Control	Flashing Signal/Blinker	Greviously Injured	0.0
1	Agra	Traffic Control	Flashing Signal/Blinker	Minor Injury	0.0
2	Agra	Traffic Control	Flashing Signal/Blinker	Persons Killed	0.0
3	Agra	Traffic Control	Flashing Signal/Blinker	Total Injured	0.0
4	Agra	Traffic Control	Flashing Signal/Blinker	Total number of Accidents	0.0
...	...	...	...	...	...
9545	Vizaq	Weather	Sunny/Clear	Greviously Injured	561.0
9546	Vizaq	Weather	Sunny/Clear	Minor Injury	252.0
9547	Vizaq	Weather	Sunny/Clear	Persons Killed	176.0
9548	Vizaq	Weather	Sunny/Clear	Total number of Accidents	1207.0
9549	Vizaq	Weather	Sunny/Clear	Total Injured	813.0

9550 rows × 5 columns

Step 2 : Deciding which data parameter is relevant according to the problem

In the case of road accidents, not all available data may directly explain the causes. For example, parameters like the cause category are very relevant because they show why accidents happen, while other details such as vehicle color may not contribute much to solving the problem. Choosing the right parameter ensures that the analysis focuses on meaningful information, avoids unnecessary complexity, and leads to more accurate conclusions and solutions.

Step 3 : Selecting a Chart Type

Choosing the correct chart is important because it determines how well the data can be understood and interpreted. The right chart makes patterns, comparisons, or proportions easier to see, while the wrong chart can confuse the audience or even hide the real meaning of the data. For example, a pie chart is best when showing proportions of a whole, such as the share of accident causes, while a bar chart is better for comparing exact numbers across

categories. By matching the chart type to the purpose of the analysis, the data becomes clearer, more accurate, and more useful for decision-making.

Pie Chart

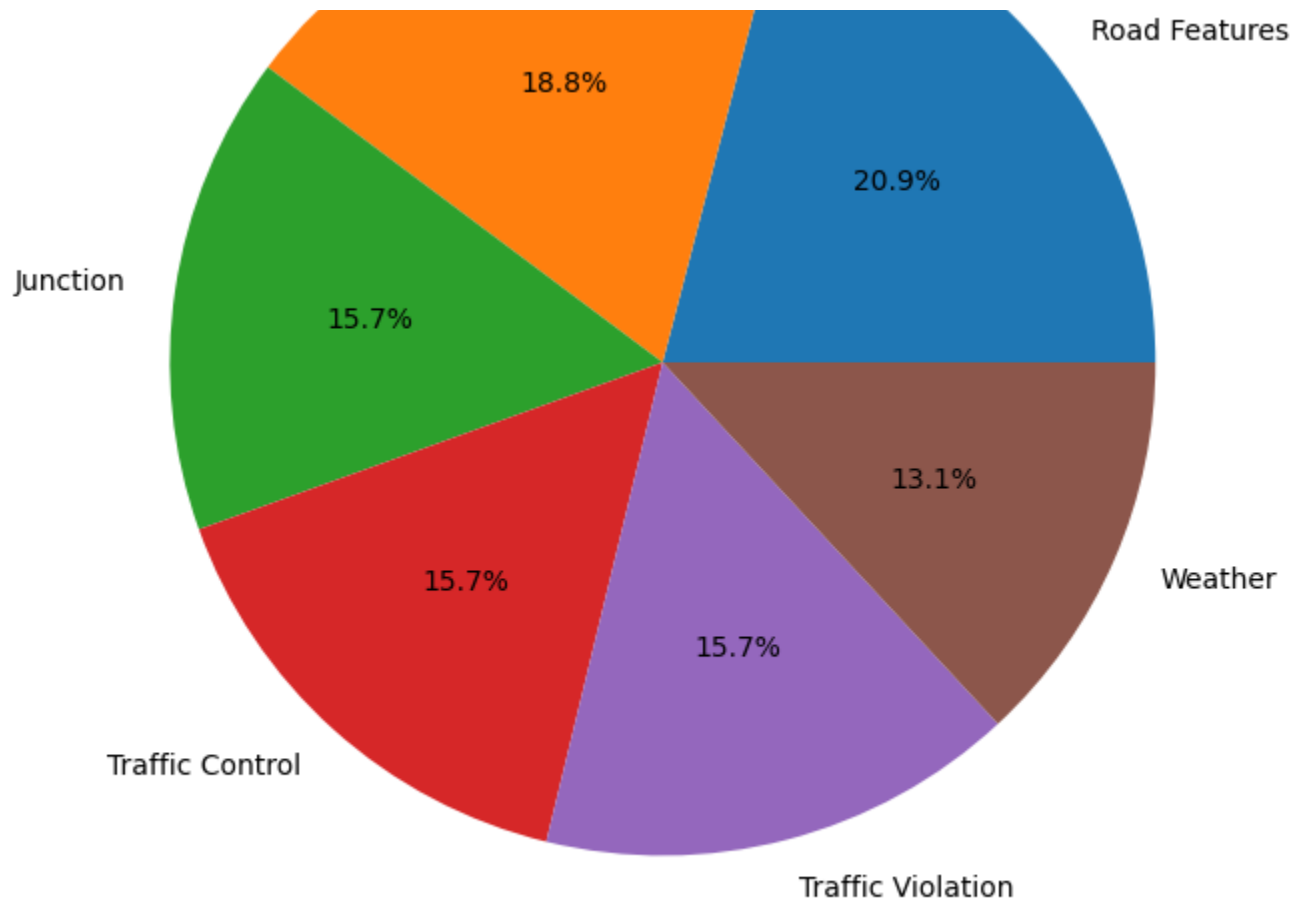
The pie chart is effective for this type of visualization because it clearly shows how road accident causes are divided into different categories. By using percentages, it is easy to compare the size of each category and quickly identify which cause contributes the most and which ones are less common. This makes the data more understandable at a glance, especially when the goal is to highlight the proportion of each cause rather than the exact numbers.

```
1 # Count values in 'Cause category'
2 cause_counts = df['Cause category'].value_counts()
3
4 # Create a pie chart
5 cause_counts.plot.pie(figsize=(8, 8), autopct='%1.1f%%')
6
7 # Add title
8 plt.title('Distribution of Road Accident Cause Categories')
9
10 # Remove y-axis label
11 plt.ylabel('')
12
13 # Show chart
14 plt.show()
```

Distribution of Road Accident Cause Categories

Impacting Vehicle/Object





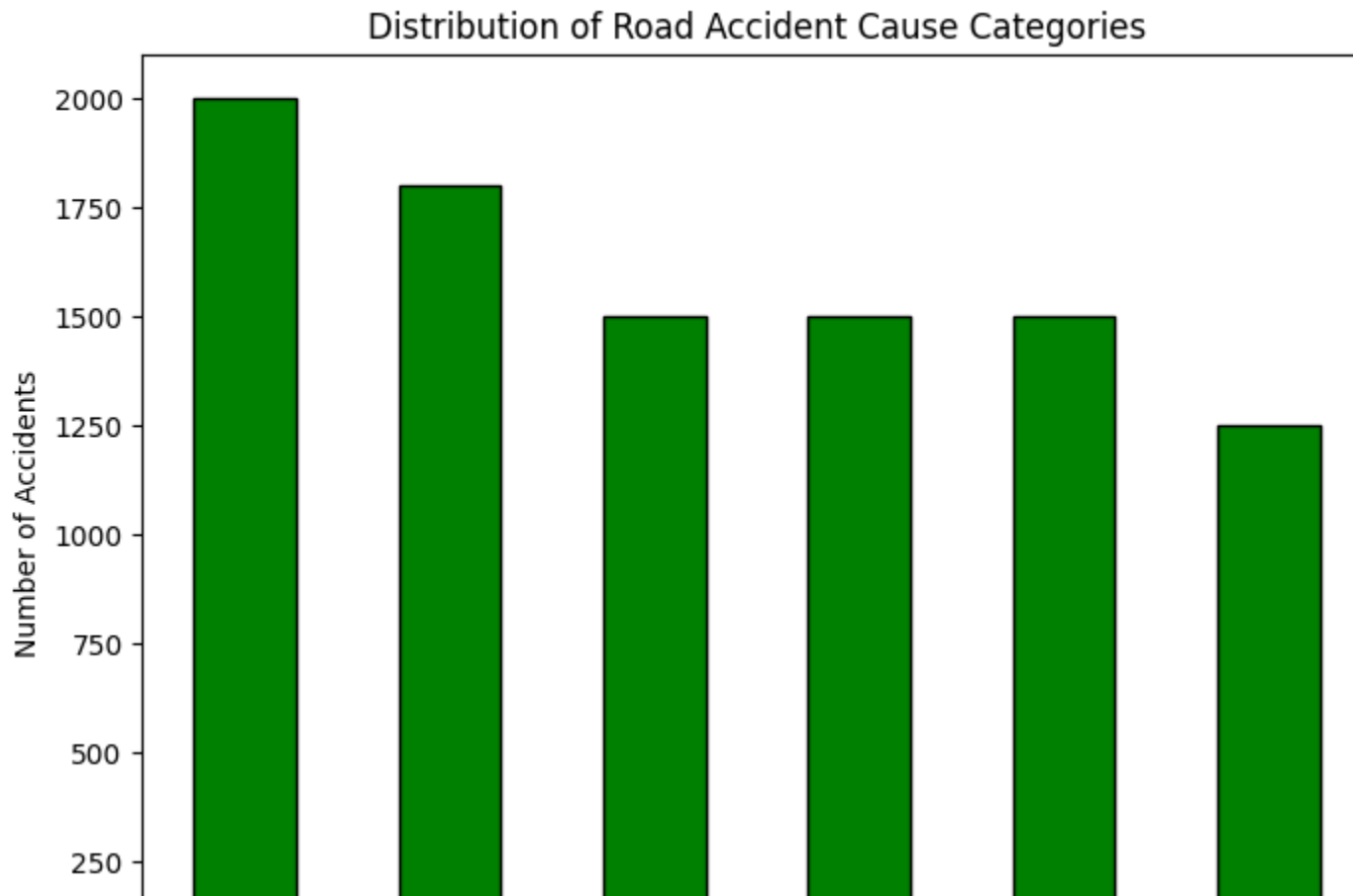
Bar Chart

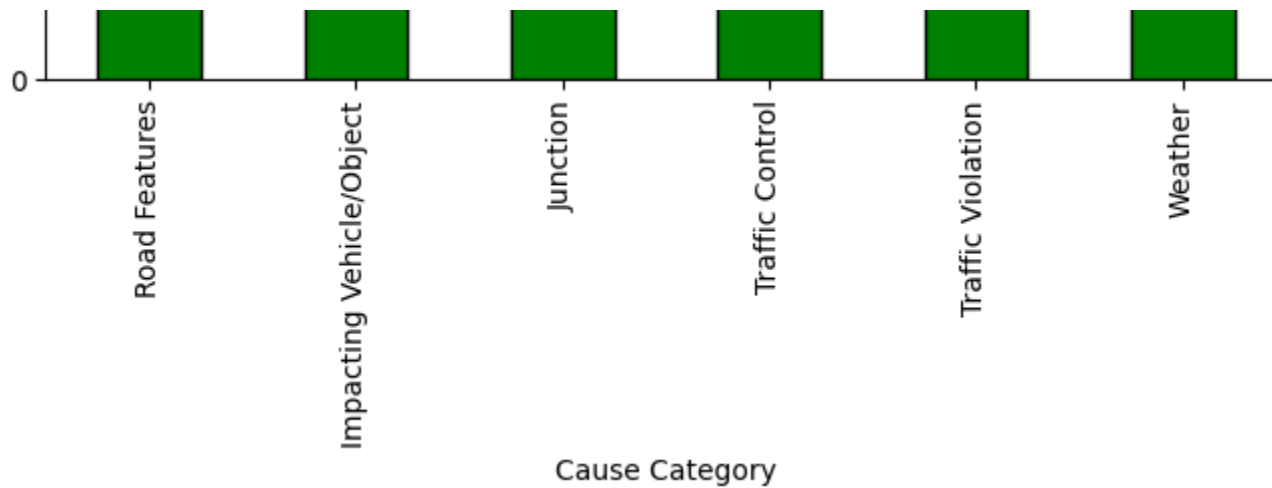
The bar chart is effective for this type of visualization because it shows the exact number of accidents for each cause category. Unlike the pie chart, it makes it easier to compare the size of each category side by side, especially when there are many categories. The height of the bars gives a clear picture of which causes are more frequent and which are less common, making the data straightforward to interpret.

```
1 # Count values in the 'Cause category' column
```

```
2 cause_counts = df['Cause category'].value_counts()
```

```
2 cause_counts = df[ 'Cause Category' ].value_counts()
3
4 # Plot a bar chart
5 cause_counts.plot.bar(figsize=(8, 6), color='green', edgecolor='black')
6
7 # Add labels and title
8 plt.title('Distribution of Road Accident Cause Categories')
9 plt.xlabel('Cause Category')
10 plt.ylabel('Number of Accidents')
11
12 # Show chart
13 plt.show()
```

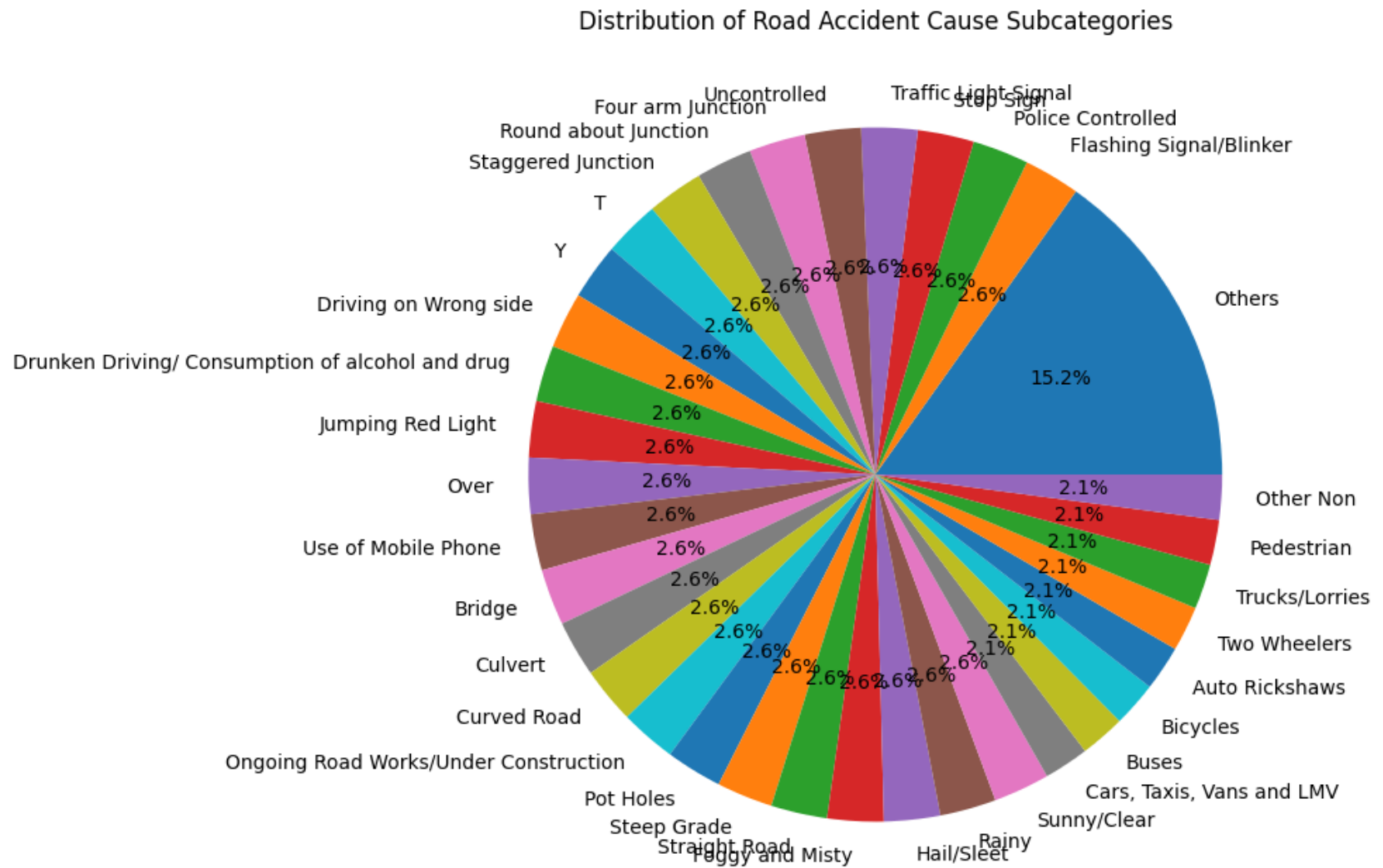




Task 1.

Using the dataset visualize the cause subcategory using the pie chart and bar chart.

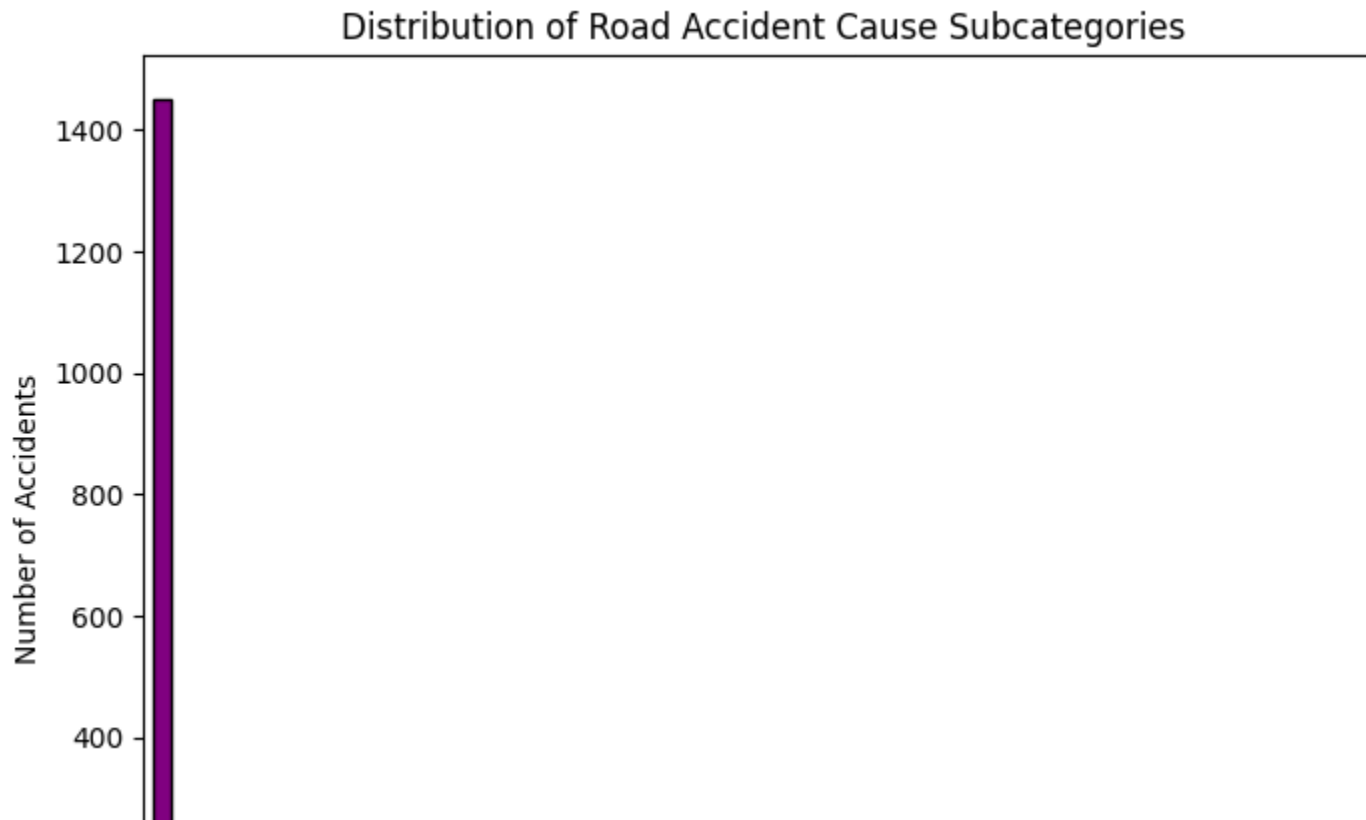
```
1 #Code here
2 ##PIE CHART
3 # Count values in 'Cause Subcategory'
4 cause_counts = df['Cause Subcategory'].value_counts()
5
6 # Create a pie chart
7 cause_counts.plot.pie(figsize=(8, 8), autopct='%1.1f%%')
8
9 # Add title
10 plt.title('Distribution of Road Accident Cause Subcategories')
11
12 # Remove y-axis label
13 plt.ylabel('')
14
15 # Show chart
16 plt.show()
```

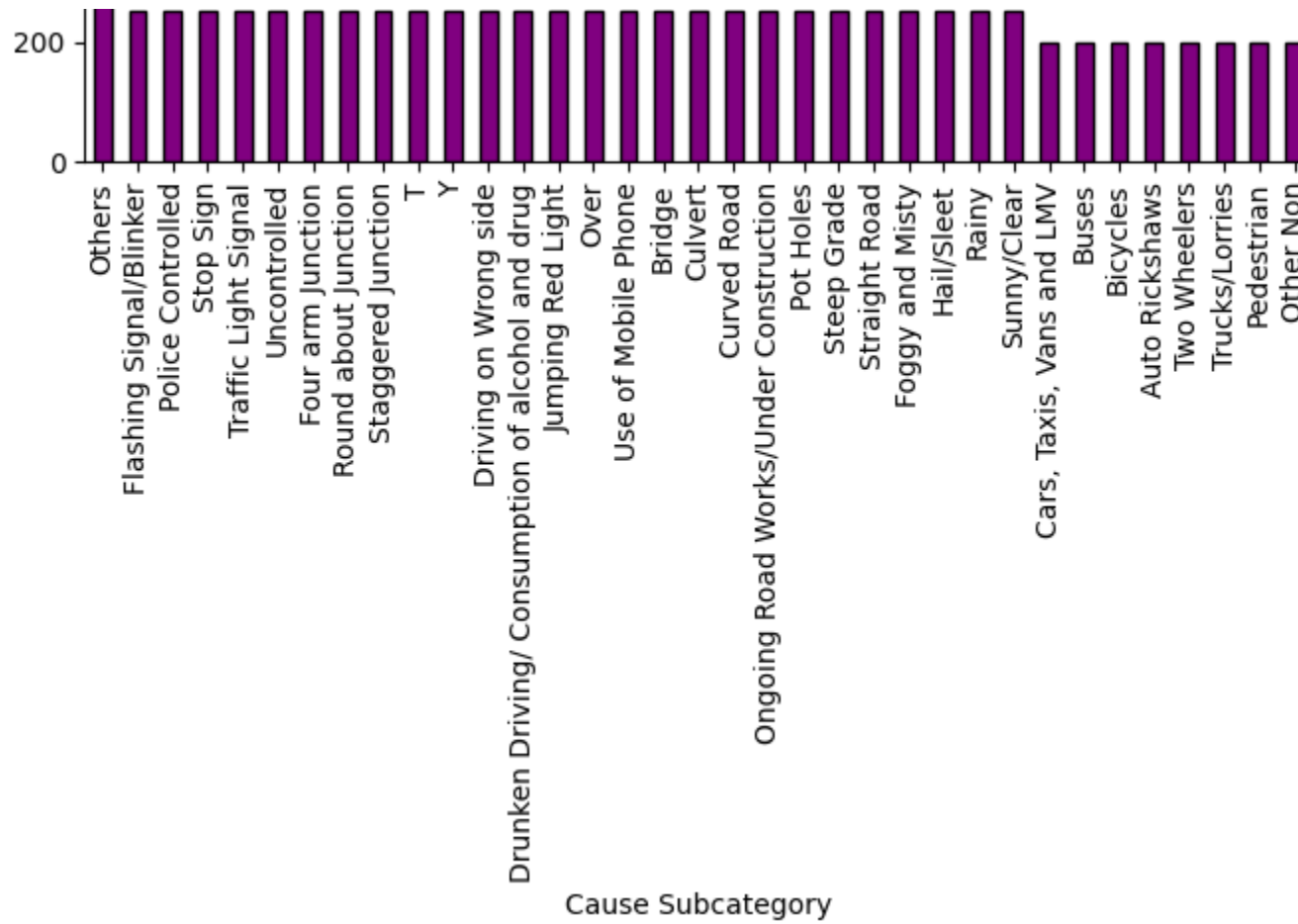



1

2. #BAR CHART

```
2 ##BARCHART
3 # Count values in the 'Cause Subcategory' column
4 cause_counts = df['Cause Subcategory'].value_counts()
5
6 # Plot a bar chart
7 cause_counts.plot.bar(figsize=(8, 6), color='purple', edgecolor='black')
8
9 # Add labels and title
10 plt.title('Distribution of Road Accident Cause Subcategories')
11 plt.xlabel('Cause Subcategory')
12 plt.ylabel('Number of Accidents')
13
14 # Show chart
15 plt.show()
```



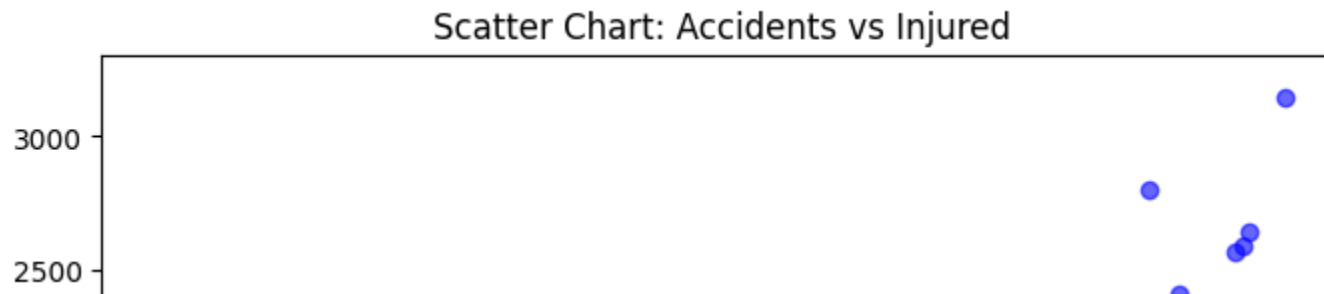


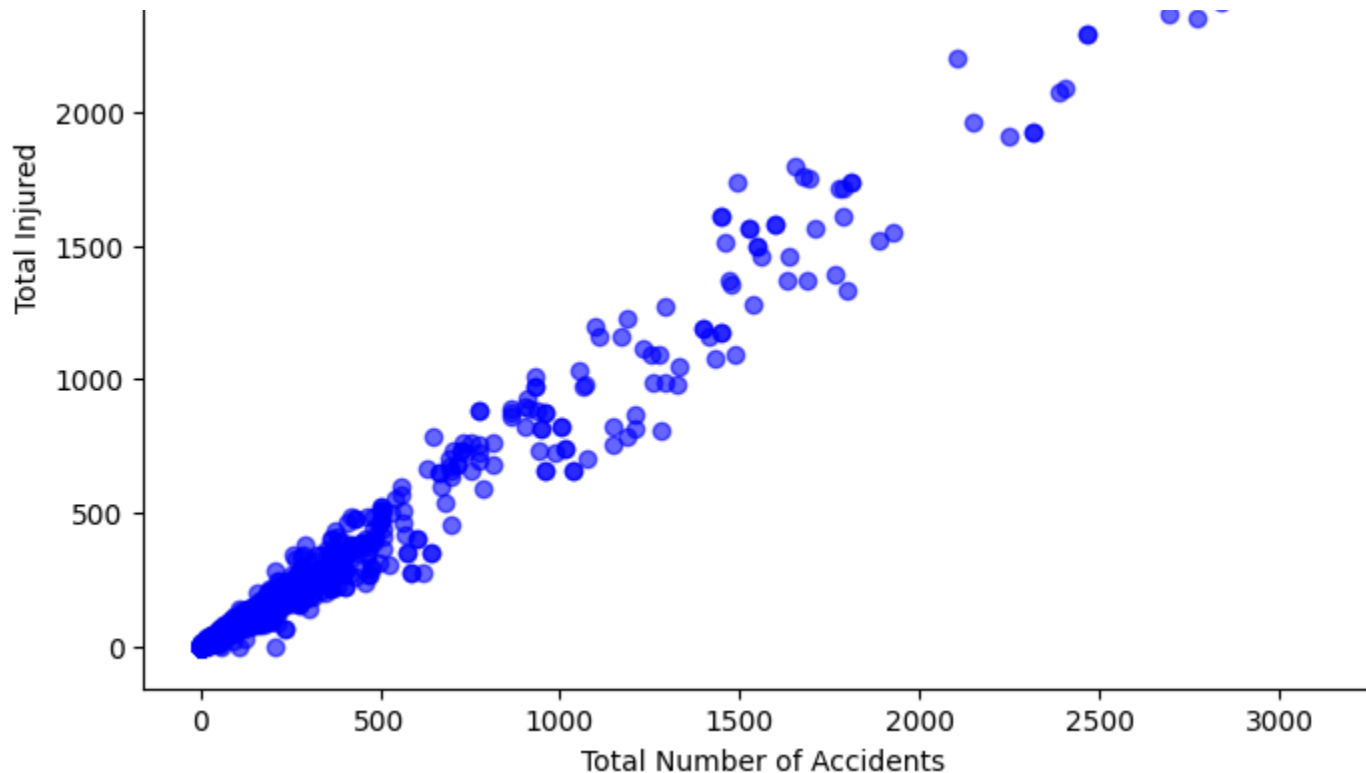
Scatter Chart

A scatter chart is best when you want to show the relationship between two numerical parameters. In your data, the "Count" column is numerical, so it should be one axis. The other axis can be a numerical transformation, such as grouping by "Cause category" or "Outcome of Incident" (converted to counts). For example, you can compare "Total number of Accidents" vs. "Total Injured" across different cities. This helps show whether more accidents always lead to more injuries.

```
1 # Filter dataset to only include rows with 'Total number of Accidents' and 'Total Injured'
```

```
2 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
3 injured = df[df['Outcome of Incident'] == 'Total Injured']
4
5 # Merge on city and cause to align values
6 scatter_data = accidents.merge(
7     injured,
8     on=['Million Plus Cities', 'Cause category', 'Cause Subcategory'],
9     suffixes=('_Accidents', '_Injured')
10 )
11
12 # Plot scatter chart
13 plt.figure(figsize=(8, 6))
14 plt.scatter(
15     scatter_data['Count_Accidents'],
16     scatter_data['Count_Injured'],
17     alpha=0.6,
18     color='blue'
19 )
20
21 # Add labels and title
22 plt.title('Scatter Chart: Accidents vs Injured')
23 plt.xlabel('Total Number of Accidents')
24 plt.ylabel('Total Injured')
25
26 # Show chart
27 plt.show()
```





The scatter chart shows how the total number of accidents relates to the total number of injured. If the points go upward as accidents increase, it means more accidents usually lead to more injuries. Points that stand out show unusual cases where the number of injuries does not match the number of accidents.

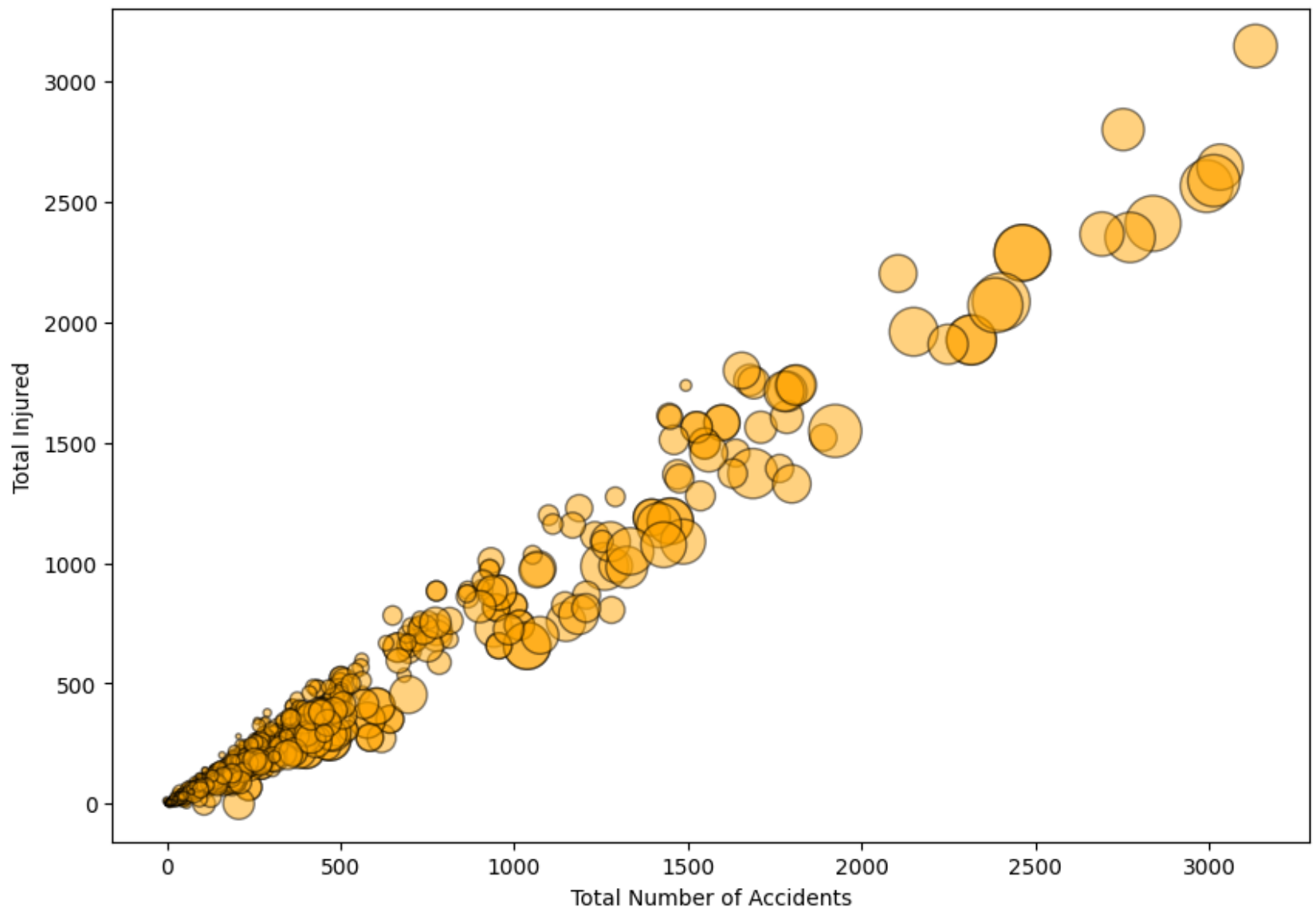
Bubble Chart

A bubble chart adds a third dimension by using bubble size. Here, the x-axis and y-axis can still be numerical values like “Total number of Accidents” and “Total Injured”, while the bubble size can represent “Persons Killed” or “Minor Injury” counts. You can also use “Million Plus Cities” or “Cause category” as grouping labels to distinguish bubbles by color.

```
1 # Filter dataset for needed outcomes
```

```
2 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
3 injured = df[df['Outcome of Incident'] == 'Total Injured']
4 killed = df[df['Outcome of Incident'] == 'Persons Killed']
5
6 # Merge data on city and cause to align values
7 bubble_data = accidents.merge(
8     injured,
9     on=['Million Plus Cities', 'Cause category', 'Cause Subcategory'],
10     suffixes=('_Accidents', '_Injured')
11 ).merge(
12     killed,
13     on=['Million Plus Cities', 'Cause category', 'Cause Subcategory']
14 )
15
16 # Plot bubble chart
17 plt.figure(figsize=(10, 7))
18 plt.scatter(
19     bubble_data['Count_Accidents'],
20     bubble_data['Count_Injured'],
21     s=bubble_data['Count'], # bubble size = persons killed
22     alpha=0.5,
23     c='orange',
24     edgecolors='black'
25 )
26
27 # Add labels and title
28 plt.title('Bubble Chart: Accidents vs Injured (Bubble Size = Persons Killed)')
29 plt.xlabel('Total Number of Accidents')
30 plt.ylabel('Total Injured')
31
32 # Show chart
33 plt.show()
```

Bubble Chart: Accidents vs Injured (Bubble Size = Persons Killed)



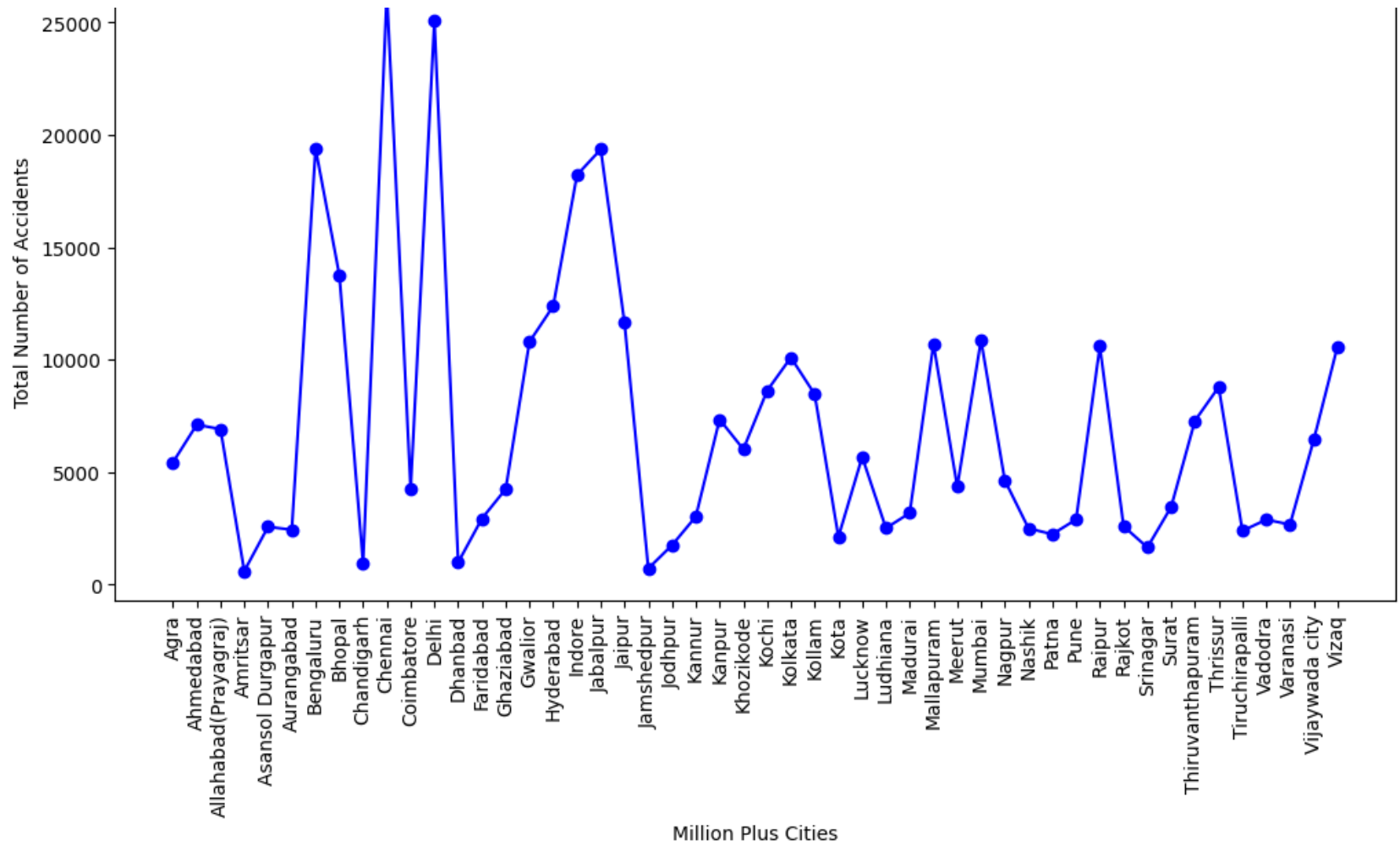
The bubble chart shows the link between the total number of accidents (x-axis) and the total number of injured (y-axis), while the bubble size represents the number of persons killed. Bigger bubbles mean more deaths in that category or city. This makes it easy to see not only how accidents relate to injuries but also where the impact is more severe in terms of fatalities.

Line Chart A line chart is useful for showing trends over time. It connects data points with lines, making it easy to see increases, decreases, or patterns across a timeline or sequence.

A line chart is effective for showing changes in road accidents or injuries across a period. For example, plotting the “Total number of Accidents” by months or years can reveal whether cases are rising, falling, or staying steady. This helps in spotting patterns and understanding the overall trend rather than just looking at single values.

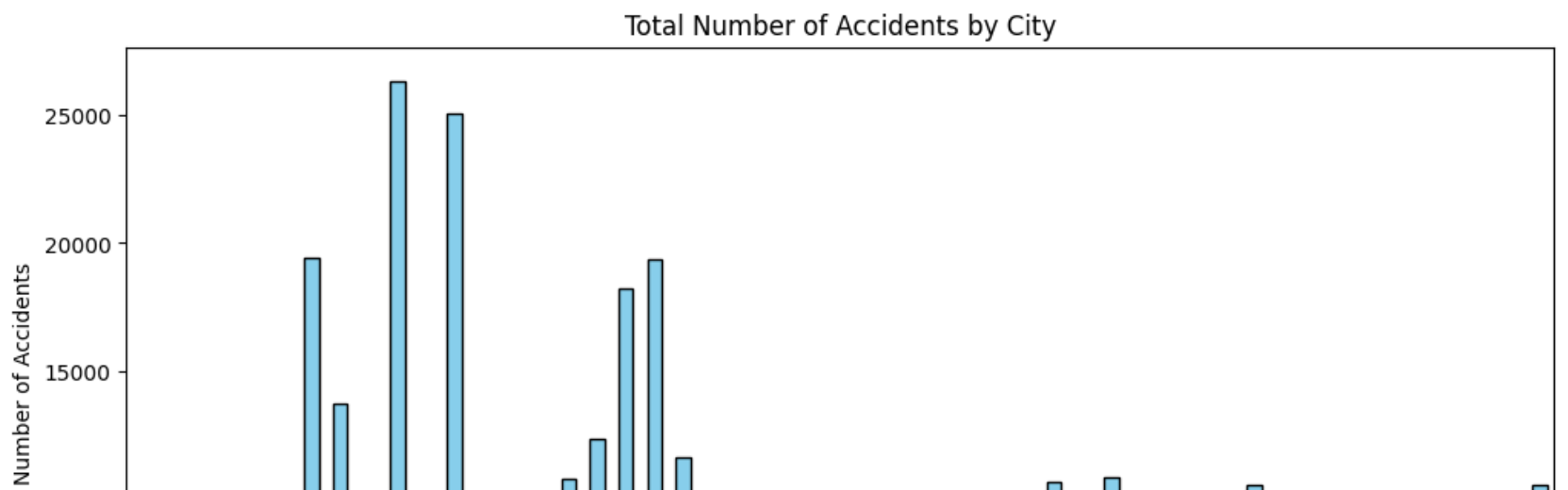
```
1 # Filter only 'Total number of Accidents'
2 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
3
4 # Group by City and sum counts
5 city_trend = accidents.groupby('Million Plus Cities')['Count'].sum()
6
7 # Plot line chart
8 plt.figure(figsize=(12, 6))
9 plt.plot(city_trend.index, city_trend.values, marker='o', linestyle='-', color='blue')
10
11 # Add labels and title
12 plt.title('Total Number of Accidents by City')
13 plt.xlabel('Million Plus Cities')
14 plt.ylabel('Total Number of Accidents')
15
16 # Rotate x-axis labels for readability
17 plt.xticks(rotation=90)
18
19 # Show chart
20 plt.show()
```

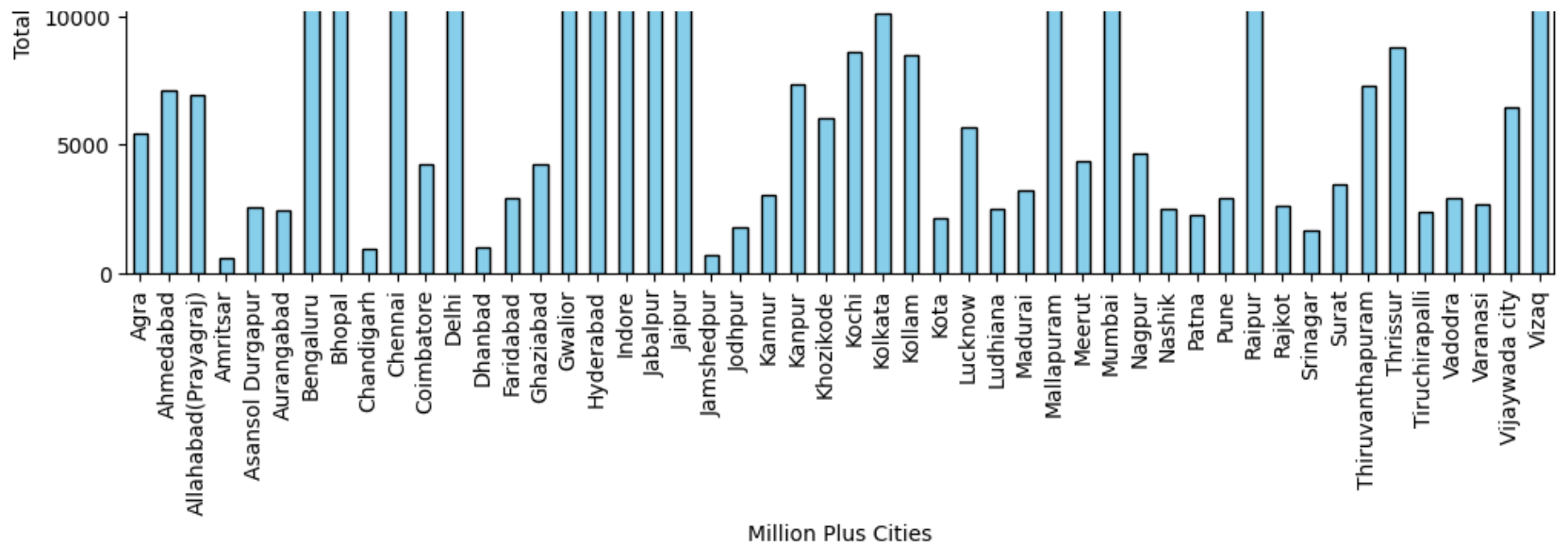
Total Number of Accidents by City



Column Chart A column chart uses vertical bars to compare values across categories. It is effective when you want to highlight differences in size or frequency between groups.

```
1 # Filter only 'Total number of Accidents'
2 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
3
4 # Group by City and sum counts
5 city_counts = accidents.groupby('Million Plus Cities')['Count'].sum()
6
7 # Plot column chart (vertical bars)
8 plt.figure(figsize=(12, 6))
9 city_counts.plot(kind='bar', color='skyblue', edgecolor='black')
10
11 # Add labels and title
12 plt.title('Total Number of Accidents by City')
13 plt.xlabel('Million Plus Cities')
14 plt.ylabel('Total Number of Accidents')
15
16 # Rotate x-axis labels for readability
17 plt.xticks(rotation=90)
18
19 # Show chart
20 plt.show()
```





Bar Chart A bar chart uses horizontal bars to compare data. It is especially helpful when category names are long or when you want to easily compare values side by side.

```

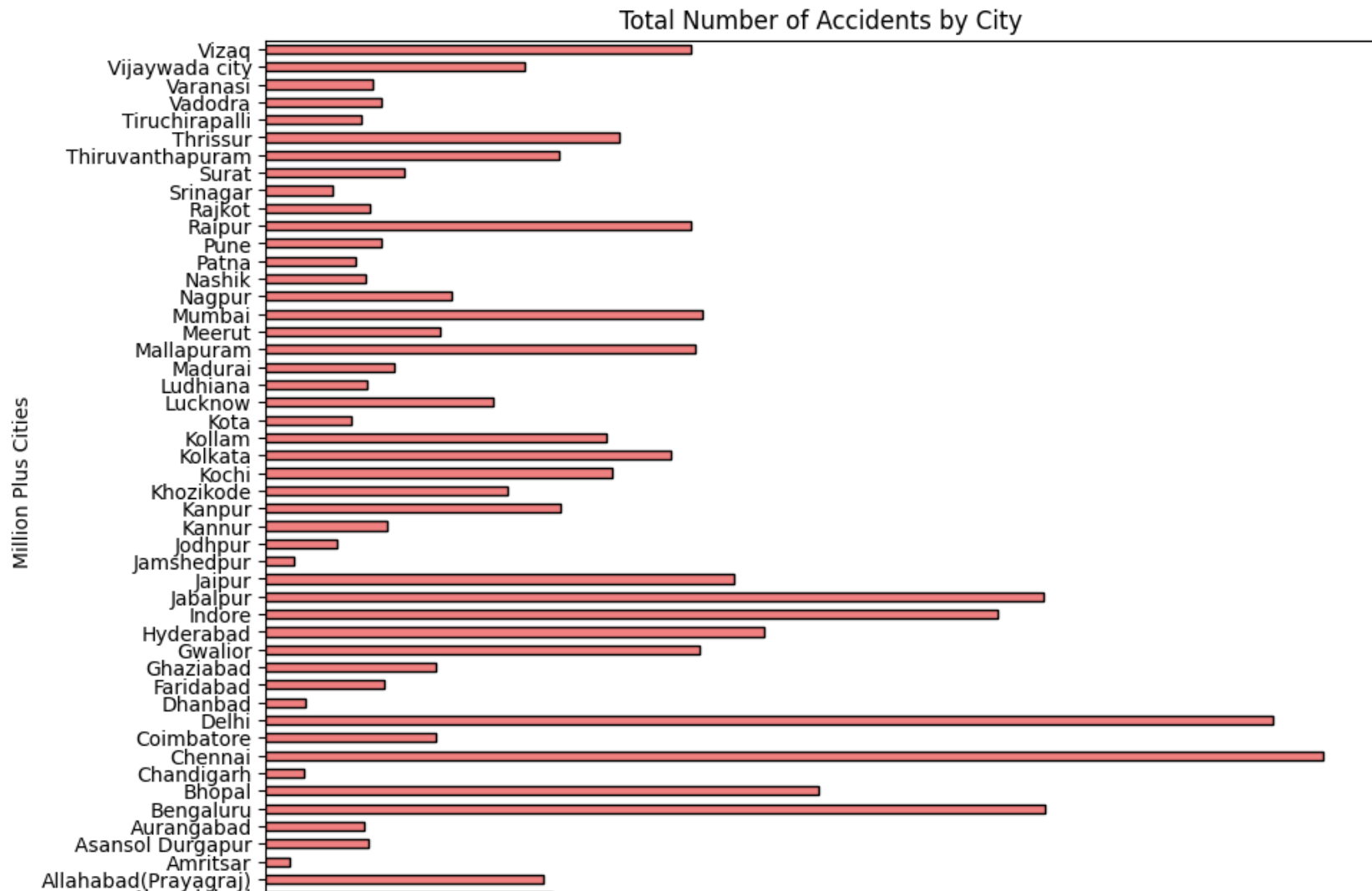
1 # Filter only 'Total number of Accidents'
2 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
3
4 # Group by City and sum counts
5 city_counts = accidents.groupby('Million Plus Cities')['Count'].sum()
6
7 # Plot bar chart (horizontal bars)
8 plt.figure(figsize=(10, 8))
9 city_counts.plot(kind='barh', color='lightcoral', edgecolor='black')

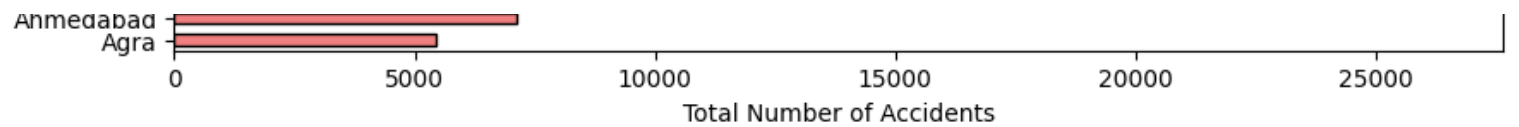
```

```

10
11 # Add labels and title
12 plt.title('Total Number of Accidents by City')
13 plt.xlabel('Total Number of Accidents')
14 plt.ylabel('Million Plus Cities')
15
16 # Show chart
17 plt.show()

```





Task 2.

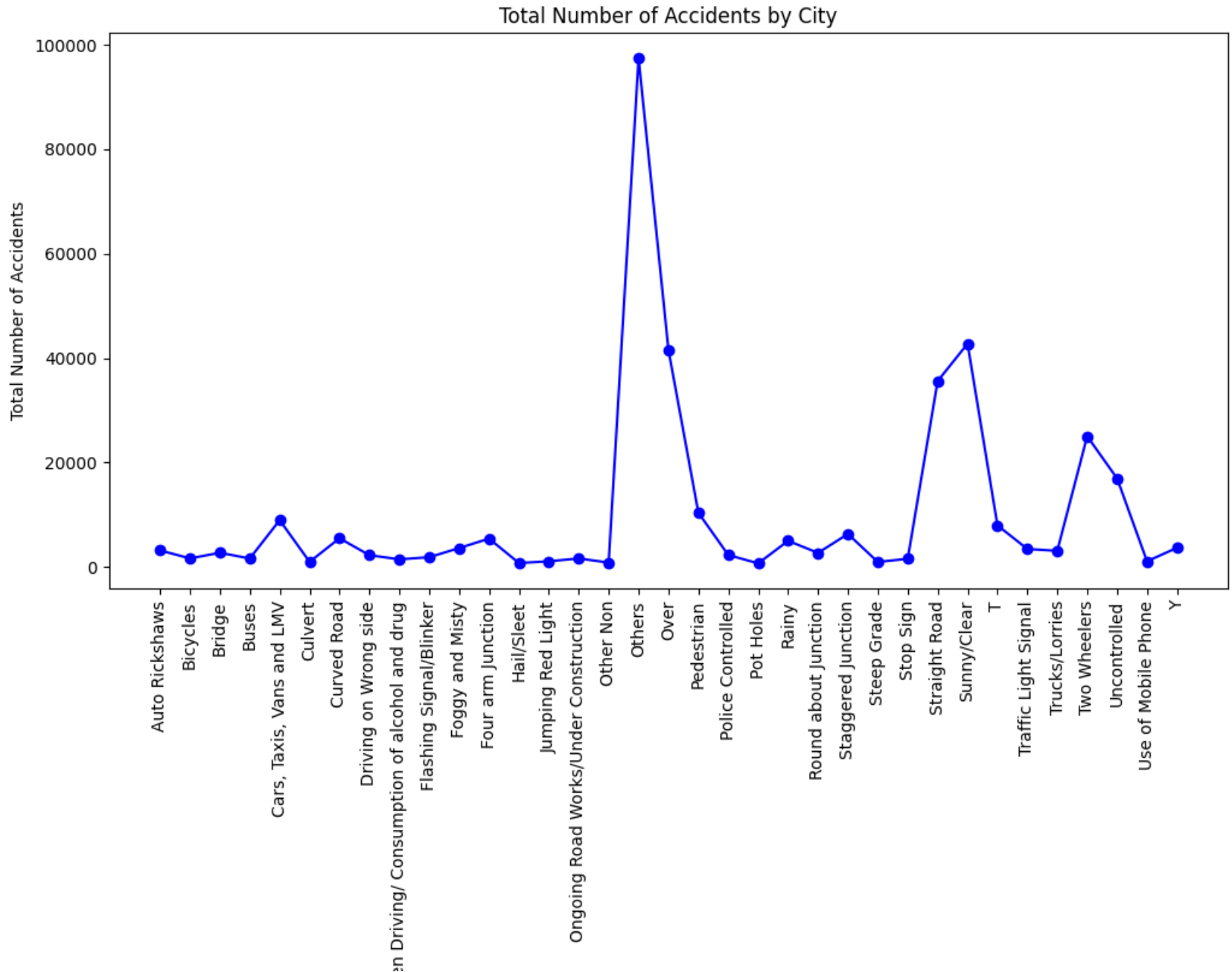
Using the line, column, and bar chart. Visualize the parameter of Cause Subcategory.

```

1 #Code here
2 ## LINE CHART
3
4 # Filter only 'Total number of Accidents'
5 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
6
7 # Group by City and sum counts
8 city_trend = accidents.groupby('Cause Subcategory')['Count'].sum()
9
10 # Plot line chart
11 plt.figure(figsize=(12, 6))
12 plt.plot(city_trend.index, city_trend.values, marker='o', linestyle='-', color='blue')
13
14 # Add labels and title
15 plt.title('Total Number of Accidents by City')
16 plt.xlabel('Cause Subcategories')
17 plt.ylabel('Total Number of Accidents')
18
19 # Rotate x-axis labels for readability
20 plt.xticks(rotation=90)
21

```

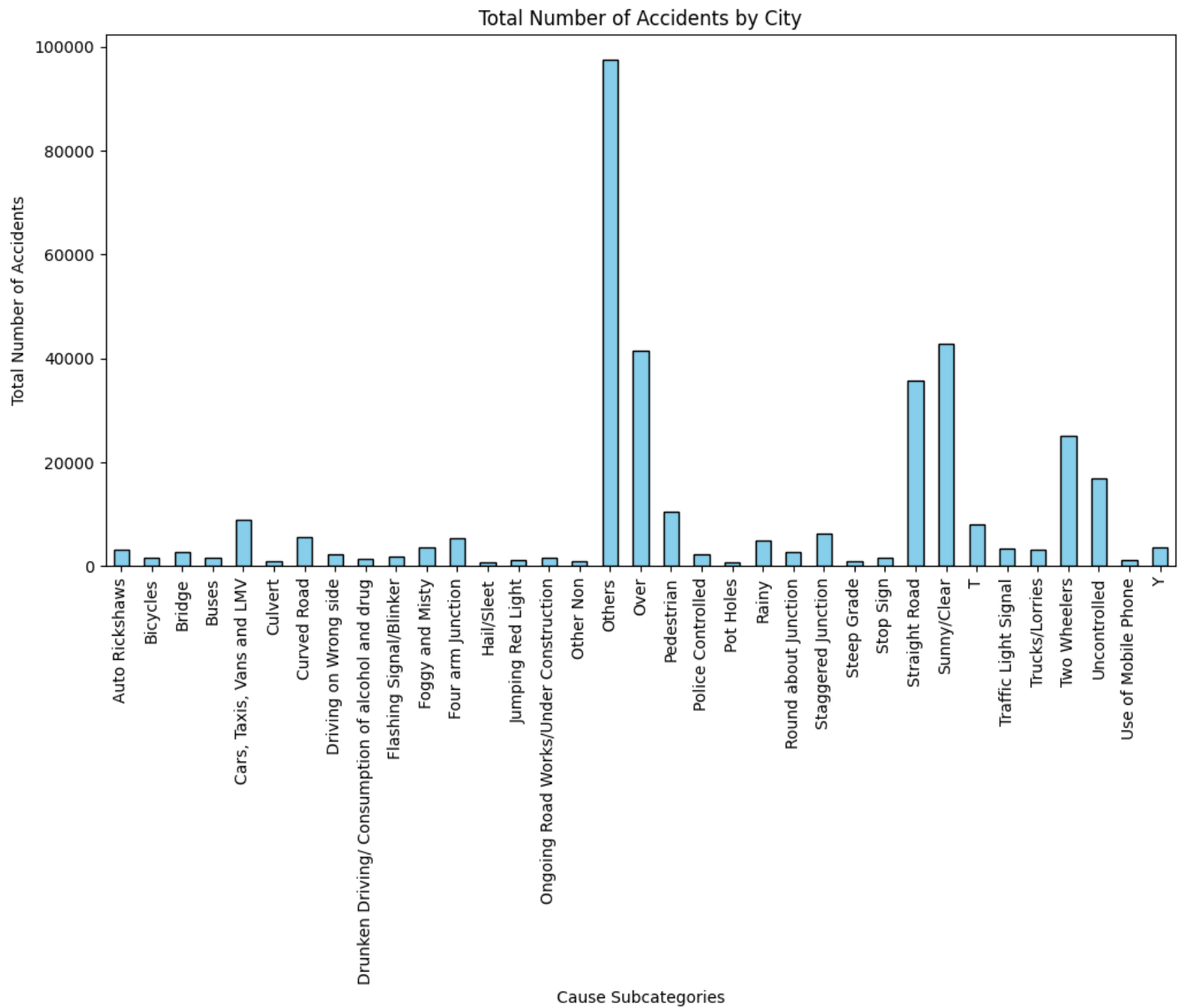
```
22 # Show chart
23 plt.show()
```



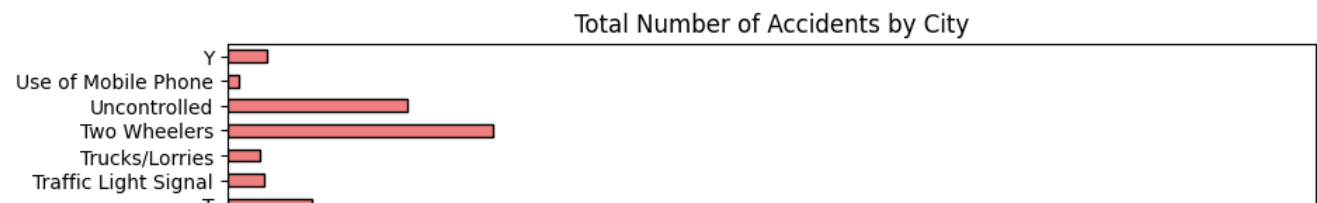
Drunk

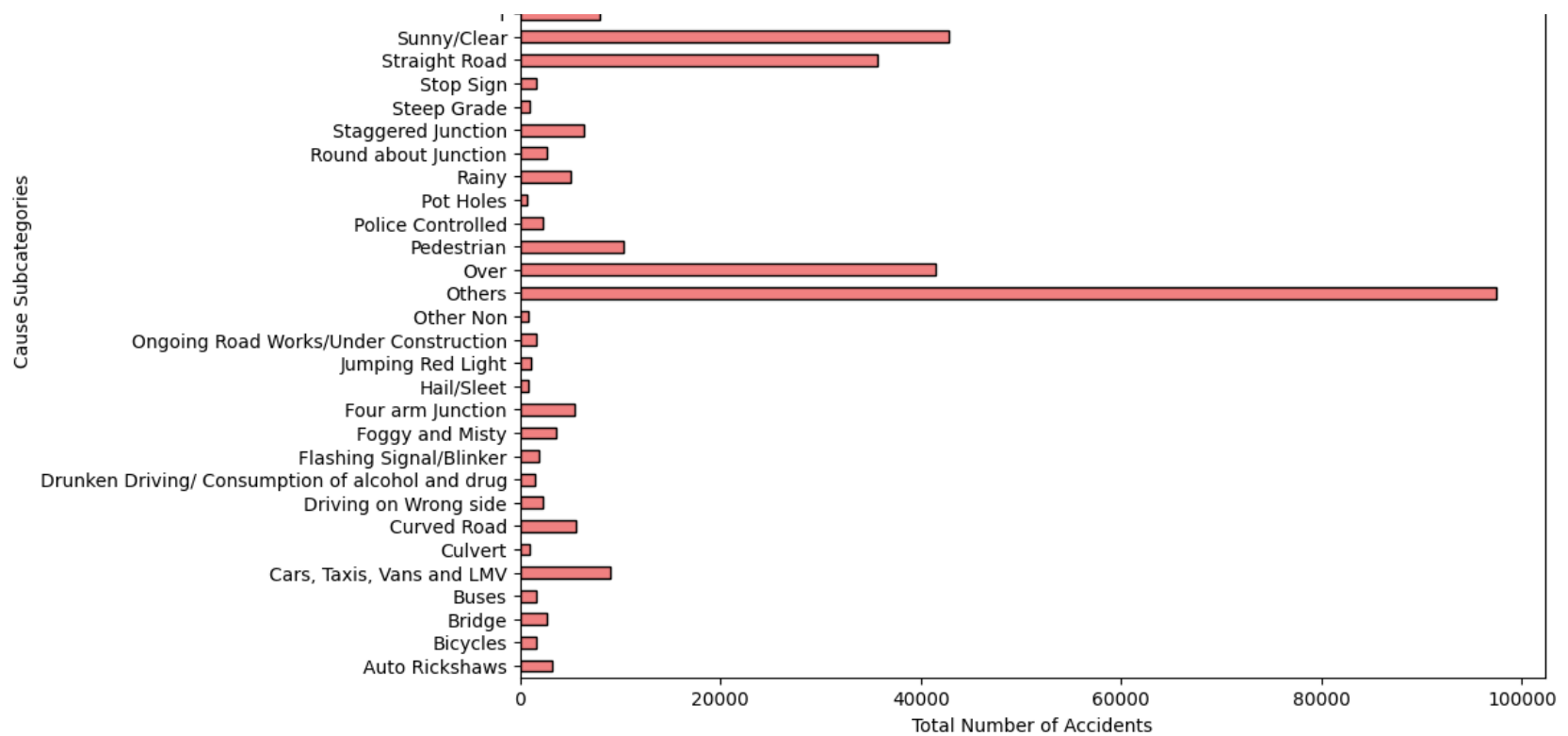
Cause Subcategories

```
1 ##COLUMN CHART
2 # Filter only 'Total number of Accidents'
3 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
4
5 # Group by City and sum counts
6 city_counts = accidents.groupby('Cause Subcategory')['Count'].sum()
7
8 # Plot column chart (vertical bars)
9 plt.figure(figsize=(12, 6))
10 city_counts.plot(kind='bar', color='skyblue', edgecolor='black')
11
12 # Add labels and title
13 plt.title('Total Number of Accidents by City')
14 plt.xlabel('Cause Subcategories')
15 plt.ylabel('Total Number of Accidents')
16
17 # Rotate x-axis labels for readability
18 plt.xticks(rotation=90)
19
20 # Show chart
21 plt.show()
```




```
1 ##BAR CHART
2
3 # Filter only 'Total number of Accidents'
4 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
5
6 # Group by City and sum counts
7 city_counts = accidents.groupby('Cause Subcategory')['Count'].sum()
8
9 # Plot bar chart (horizontal bars)
10 plt.figure(figsize=(10, 8))
11 city_counts.plot(kind='barh', color='lightcoral', edgecolor='black')
12
13 # Add labels and title
14 plt.title('Total Number of Accidents by City')
15 plt.xlabel('Total Number of Accidents')
16 plt.ylabel('Cause Subcategories')
17
18 # Show chart
19 plt.show()
```



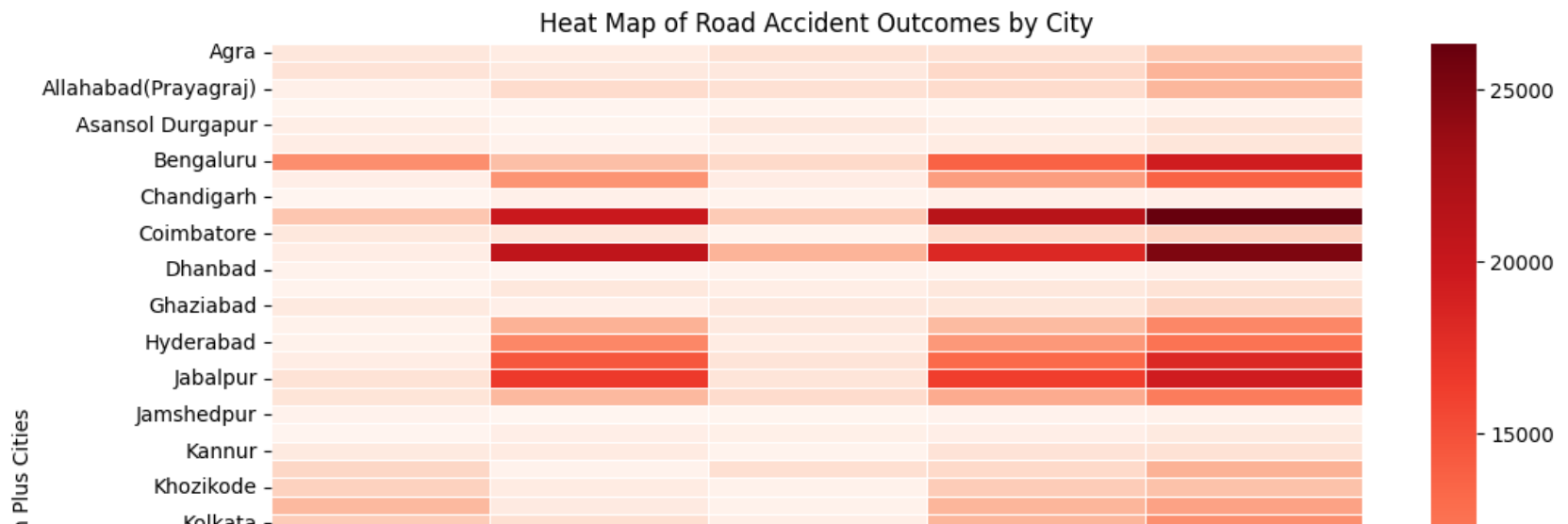


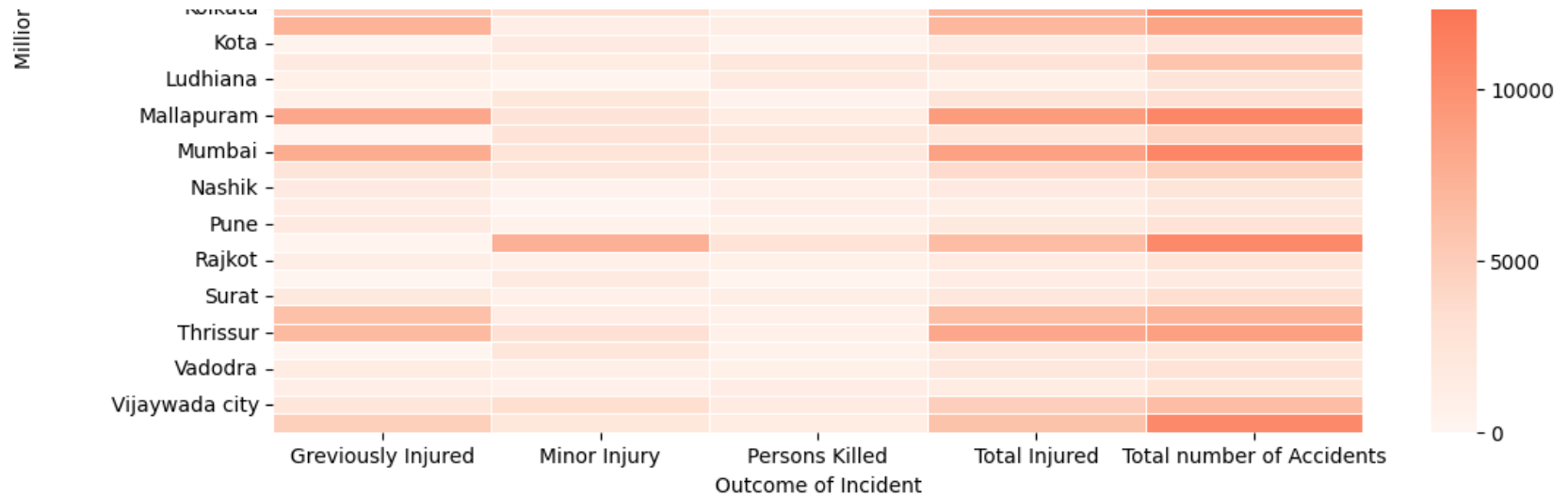
MAPS

Heat Map

```
1 import seaborn as sns
```

```
1 # Pivot table: Cities (rows) vs. Outcomes (columns)
2 heat_data = df.pivot_table(
3     index="Million Plus Cities",
4     columns="Outcome of Incident",
5     values="Count",
6     aggfunc="sum"
7 ).fillna(0)
8
9 # Plot heatmap
10 plt.figure(figsize=(12, 8))
11 sns.heatmap(heat_data, cmap="Reds", linewidths=0.5)
12
13 plt.title("Heat Map of Road Accident Outcomes by City")
14 plt.xlabel("Outcome of Incident")
15 plt.ylabel("Million Plus Cities")
16 plt.show()
```





The heat map shows the intensity of accident outcomes across different cities. Darker colors represent higher numbers, making it easy to see which cities report more accidents, injuries, or fatalities. This type of map is effective for spotting patterns and quickly identifying cities that need more attention in terms of road safety.

Tree Map

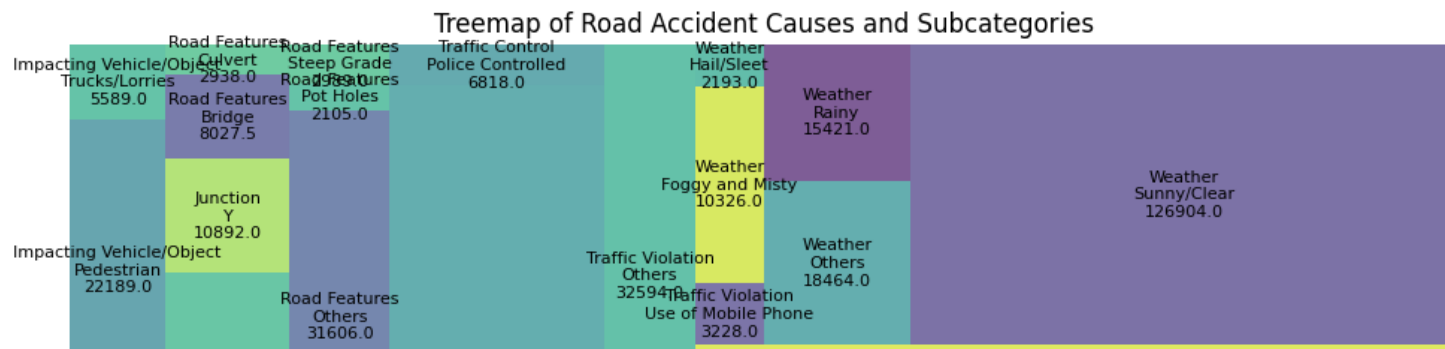
The treemap breaks down accidents by cause category and subcategory. Each rectangle's size represents the number of cases, so larger rectangles mean more frequent causes. This visualization is useful because it highlights which accident causes dominate overall and also shows how subcategories contribute within each main cause.

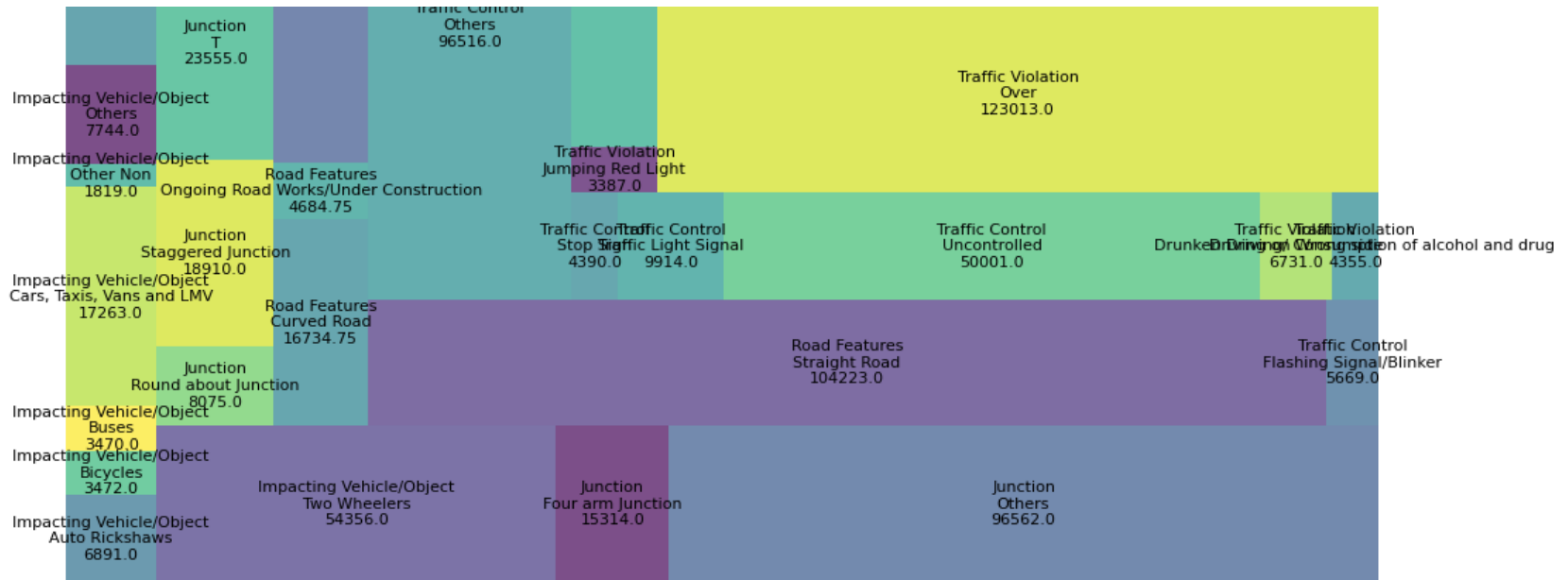
```
1 !nin install squarify
```

Requirement already satisfied: squarify in /usr/local/lib/python3.13/dist-packages (0.4.4)

```
1 import matplotlib.pyplot as plt
2 import squarify
```

```
1 # Group by Cause category and subcategory
2 treemap_data = df.groupby(["Cause category", "Cause Subcategory"])["Count"].sum().reset_index
3
4 # Prepare labels
5 labels = treemap_data["Cause category"] + "\n" + treemap_data["Cause Subcategory"] + "\n"
6
7 # Plot treemap with smaller font size
8 plt.figure(figsize=(12, 8))
9 squarify.plot(
10     sizes=treemap_data["Count"],
11     label=labels,
12     alpha=0.7,
13     text_kwargs={'fontsize':8}    # correct way to set text size
14 )
15
16 plt.title("Treemap of Road Accident Causes and Subcategories")
17 plt.axis("off")
18 plt.show()
```





TABLES VERSUS CHARTS

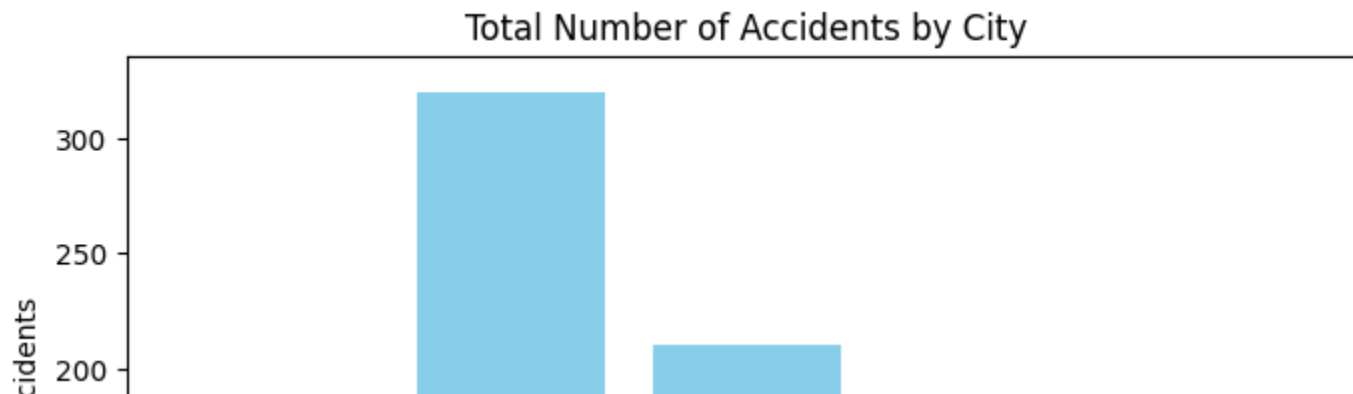
In general, charts can often convey information faster and easier to readers than tables, but in some cases a table is more appropriate. Tables should be used when the:

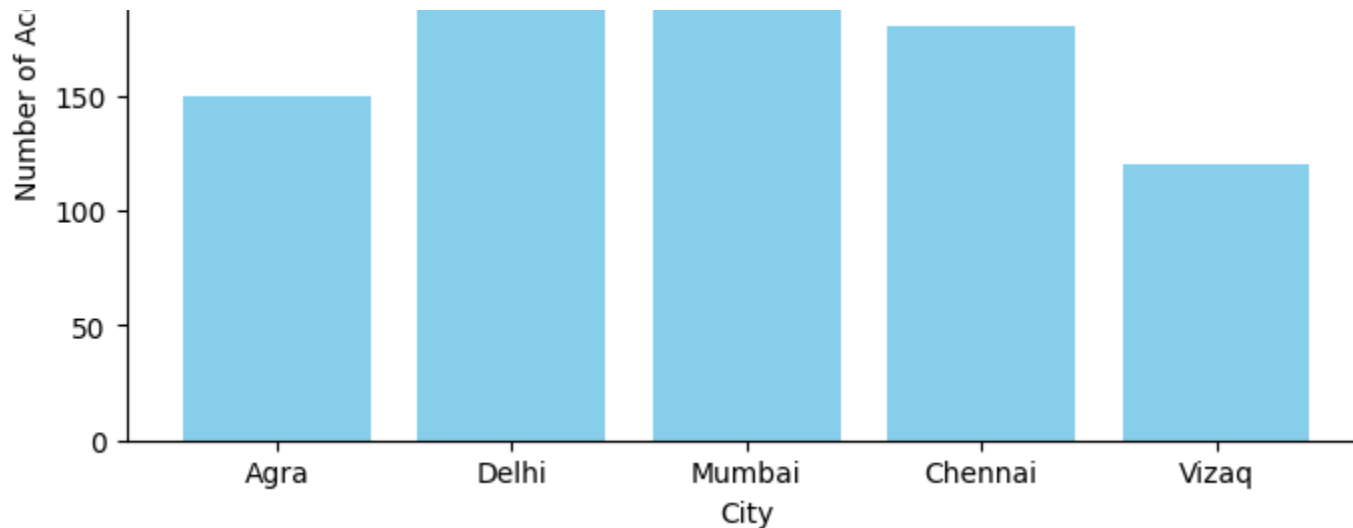
- reader needs to refer to specific numerical values.
- reader needs to make precise comparisons between different values and not just relative comparisons.
- values being displayed have different units or very different magnitudes

```
1 # Sample data (not in the Road Accidents Dataset)
2 data = {
3     "City": ["Agra", "Delhi", "Mumbai", "Chennai", "Vizaq"],
4     "Total number of Accidents": [150, 320, 210, 180, 120]
5 }
6 df_table = pd.DataFrame(data)
7
8 # Show the table
9 print("Table of Total Number of Accidents by City")
10 print(df_table)
11
12 # Plot the chart
13 plt.figure(figsize=(8, 5))
14 plt.bar(df_table["City"], df_table["Total number of Accidents"], color="skyblue")
15 plt.title("Total Number of Accidents by City")
16 plt.xlabel("City")
17 plt.ylabel("Number of Accidents")
18 plt.show()
```

Table of Total Number of Accidents by City

	City	Total number of Accidents
0	Agra	150
1	Delhi	320
2	Mumbai	210
3	Chennai	180
4	Vizaq	120





5. Supplementary Activity:

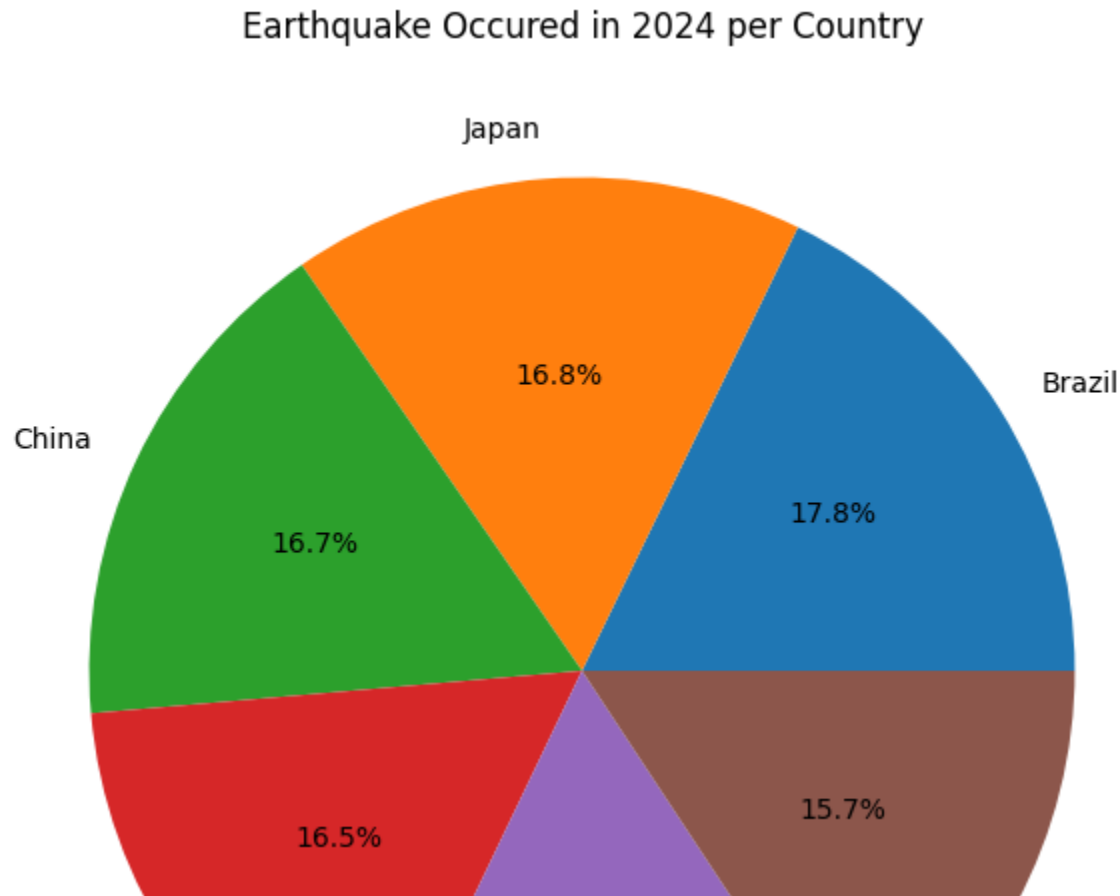
1. Identify and obtain a dataset in .csv format from reputable sources such as Kaggle or Google Dataset Search.
2. Formulate a clear problem statement that specifies what aspect of the data you aim to analyze or visualize.
3. Apply an appropriate visualization technique by selecting the chart type that best represents the dataset and effectively addresses the stated problem.
4. Provide a comprehensive explanation of the chosen chart, justifying why it is suitable for representing the data and how it contributes to understanding the problem.

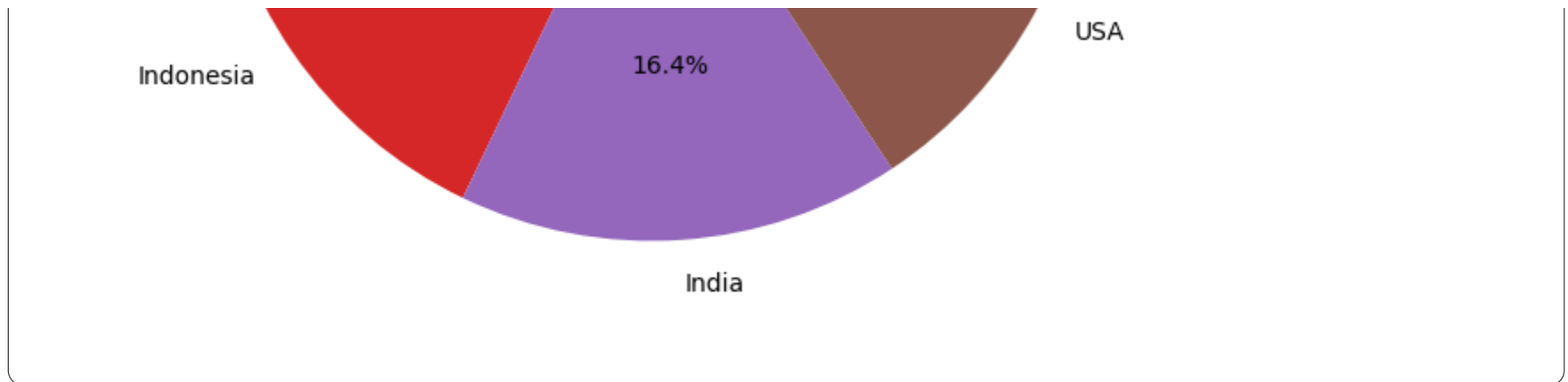
```
1 df = pd.read_csv("/content/natural_disasters_2024.csv")
```

```
1 #Code here
2 # Specifying disaster type
3 disaster = df[df['Disaster_Type'] == 'Earthquake']
4
5 # Location of disaster Earthquake
6 locCount = disaster['Location'].value_counts()
```



```
7
8 # Create a pie chart
9 locCount.plot.pie(figsize=(8, 8), autopct='%1.1f%%')
10
11 # Add title
12 plt.title('Earthquake Occured in 2024 per Country')
13
14 # Remove y-axis label
15 plt.ylabel('')
16
17 # Show chart
18 plt.show()
```





SUPPLEMENTARY ANALYSIS:

- Based from what we have done in the supplementary, the QUESTION that we have
- ✓ is: **How many percentage of earthquake disaster per country did happen on the Year 2024?**

since our data set is the amount of natural disasters that had happened in 2024, including other kind of disasters, So what we want is to specifically look for only just the earthquake disaster and to look for what countries have them and look for the percentage that each country did the earthquake happen. Basically, we just want to know How many percentage of the total earthquake did happen per country.

Now we used the Pie Chart for this because we want to find to total distribution of the earthquake per country. This helps the users/other viewer to see the distribution of the total earthquake has for each country very easily, because we could Identify who or what country has the MOST earthquake from all or the total happenstance of this kind of disaster.

6. Conclusion/Learnings/Analysis:

*Type it here

Overall, with this lesson or activity, I now know that we could actually just use python to show the different graphs and datas instead of the use of like, Excel or other software. This is new to me because, I do not see the backend? or the code from other sites on how they show the data or how they were able to show the data/graph to the viewer, I just know that It is there, maybe a PNG or A screenshot of some sort.

In terms of graphs, since we have already discussed it and its uses back in prelim, there is not that much new knowledge, just the codes on how they work and how they look for the data from like a csv format.

In my opinion, the use of the heatmap is not a good use of a chart, and I do not know where and what is the appropriate use of it because there are other charts or graphs that can be used to show its data, and color is kind of subjective? like, on different monitors, its color varies, can be more the same as the last, or more specific (8bit vs Xbit color).

For my conclusion, the use of this charts and graphs trough the use of python allows the user to be like, lock the data so it does not change, unlike the use of excel, where you can change the data from the tables and it simultaneously change the graph with how you change it.

